FICHE MODULE 1

Comprendre la machine

L'agent ne devine pas : il prolonge ce qu'on lui donne. Cette fiche reprend l'ouverture et le module 1 dans l'ordre des slides : ce que fait vraiment un LLM, et ce qu'est un agent de code.

Vibe coding vs AI-driven dev

Ouverture · slide « Vibe coding vs AI-driven dev »

	Vibe coding	AI-driven dev
Logique	explorer vite	produire de façon fiable
Marche bien pour	spike, prototype, script jetable	code livré, relu, maintenu
Réflexe	on tente, on observe, on jette	on cadre, on contrôle, on capitalise

> Les deux sont utiles : **viber pour explorer, piloter pour livrer.**

Sortie subie ou sortie pilotée ?

Ouverture · slide « Sortie subie ou sortie pilotée ? »

Une scène familière : « Ajoute la pagination sur l'endpoint /bookings. » L'agent part sur de l'offset avec un helper maison ; on répond « on utilise des curseurs » ; il invente son propre format de page ; on rappelle les tests ; il écrit des tests qui mockent tout. Quatre allers-retours : une **sortie subie**.

Chaque relance révélait une information disponible **dès le départ** :

Ce qu'on a dit	Ce qu'il fallait donner
« ajoute la pagination »	un **objectif** et un **critère de fin**
« on utilise des curseurs »	les **conventions** du projet
« le format de page commun »	les **invariants**
« les tests »	les **commandes de vérification**

> Pas de prompt magique : on construit l'entrée pour une **sortie pilotée**.

Six notions, une phrase chacune

Module 1 · slide « Six notions, une phrase chacune »

Notion	En une phrase
LLM	Le moteur : un réseau de neurones qui **prédit le token suivant**, à partir de tout ce qu'on lui envoie.
Token	L'unité de compte : un morceau de mot ou de code. Le modèle lit, écrit, **limite et facture** en tokens.
Context window	Tout ce que le modèle voit : le texte envoyé à chaque appel. **Ce qui n'y est pas n'existe pas.**
Harness	Tout sauf le modèle : le logiciel qui l'entoure (boucle, tools, contexte, permissions).
Tools	Les mains : lire un fichier, lancer une commande, chercher. Le modèle les **demande**, le harness les exécute.
Agent	Le modèle qui agit : placé dans une **boucle** avec des tools et un objectif, il choisit lui-même la prochaine action.

Un même produit couvre plusieurs cases : Claude Code ou Copilot sont des harness, et deviennent des agents une fois lancés avec un modèle et une tâche. On classe des rôles, pas des marques.

Le texte découpé en tokens

Module 1 · slide « Le texte découpé en tokens »

Le modèle ne lit pas des mots, mais des tokens. Découpage réel avec le tokenizer o200k_base (GPT-4o, GPT-5) ; les autres modèles découpent un peu différemment, l'ordre de grandeur reste le même.

Texte	Tokens
« Ajoute la pagination sur l'endpoint /bookings. » (6 mots)	12
« Add pagination to the /bookings endpoint. » (6 mots)	9
« anticonstitutionnellement » (1 mot)	5
`return res.status(404).json({ error: "Not found" });`	15

- Les mots **courants** tiennent en un token, les mots **rares** et les identifiants sont découpés.
- Le **français** consomme plus de tokens que l'anglais pour dire la même chose.
- Un fichier de 2 000 lignes lu par l'agent, c'est des dizaines de milliers de tokens dans la context window.

> Plus de tokens = **plus cher**, **plus lent**, et **plus de place** prise dans la context window.

Plausible ≠ vrai

Module 1 · slide « Choisir le token suivant : plausible ≠ vrai » et démo « La souris mange… »

Après `return res.status(` , le modèle tire le token suivant parmi les candidats **les plus probables** : 200, 404, 201… Le bon code de retour dépend du contrat de l'API ; s'il n'est pas dans le contexte, le modèle écrit

le plus fréquent dans ses données d'entraînement.

- **Une part de hasard** : relancer donne une autre réponse. Une réponse est un tirage, pas un calcul.
- **Chaque mot pèse** : `curseur`, `OpenAPI`, `avec tests` déplacent toute la suite.

> Un LLM produit du **plausible**, pas du **vrai**. Sans la bonne information dans le contexte, l'hallucination est le fonctionnement normal.

Quand la sortie bouge beaucoup, la bonne question n'est pas « quel modèle ? » mais « **qu'est-ce que je n'ai pas cadré ?** ».

Le modèle n'a pas de mémoire

Module 1 · slide « Le modèle n'a pas de mémoire »

À chaque appel, le harness renvoie **toute la session** au modèle.

- Une erreur du tour 2 est **relue à chaque tour** suivant.
- La « mémoire » du harness (fichiers d'instructions, notes gardées) est du texte **relu à chaque session** : une info périmée continue de peser.
- Une **nouvelle session** repart de zéro : c'est un outil, pas un échec.

Chat, workflow ou agent ?

Module 1 · slide « Chat, workflow ou agent ? »

	Chat ou complétion	Workflow LLM	Agent
Qui décide de l'étape suivante ?	l'humain	le **code** (chemin écrit à l'avance)	le **modèle**
Tools	aucun ou peu	appelés à des étapes fixes	choisis à chaque tour
Appels au modèle	1 par demande	nombre fixe	variable, inconnu à l'avance
Exemple	suggestion inline, « Tab pour accepter »	résumer chaque PR à l'ouverture	« Ajoute la pagination à /bookings, avec ses tests »

> Règle d'Anthropic (*Building effective agents*) : commencer par **la solution la plus simple** qui marche. Un workflow prévisible bat souvent un agent autonome.

Modèle ≠ harness

Module 1 · slide « Modèle ≠ assistant (le harness) »

- Le **modèle** est le moteur ; le **harness** lui donne les moyens d'agir : boucle de tools, contexte, permissions, traces.

- **Même modèle, deux harness** : des résultats très différents. Le harness pèse autant que le choix du modèle.

> Les noms changent tous les trimestres. Les notions restent.

La boucle agentique

Module 1 · slide « La boucle agentique »

Agent = modèle + tools + boucle + objectif. Le modèle n'exécute rien : il écrit une **demande d'action**, le harness l'exécute.

1. Le harness envoie la demande et la liste des tools.
2. Le modèle répond « j'appelle bash avec `npm test` ».
3. Le harness vérifie que c'est autorisé, exécute, récupère la sortie.
4. Il renvoie **tout** au modèle, plus la sortie.
5. On recommence jusqu'à une réponse **sans appel de tool**.

C'est dans le harness, pas dans le modèle, qu'on met les permissions, la sandbox et la confirmation humaine. Le meilleur critère d'arrêt est observable : « les tests de /bookings passent ».

Une trace commentée

Module 1 · slide « Une trace d'agent, commentée »

Trace simplifiée et représentative ; les chiffres sont des ordres de grandeur.

Tour	Le modèle demande	Le harness renvoie	Contexte
1	`grep: "bookings"`	6 fichiers	12 k
2	`read_file: bookings.controller.ts`	20 lignes	14 k
3	`read_file: shared/pagination/cursor.ts`	100 lignes	15 k
4	`edit: bookings.controller.ts` + test	diff appliqué, après accord	17 k
5	`bash: npm test -- bookings`	échec : `nextCursor is undefined`	19 k
6	`edit: bookings.controller.ts`	diff appliqué	20 k
7	`bash: npm test -- bookings`	succès : `12 passed`	21 k

- L'agent commence par **chercher et lire** : il découvre le code comme nous.
- Au tour 4, le harness demande l'**accord** avant d'écrire, selon ses permissions.
- Au tour 5, c'est l'**échec du test** qui ramène l'agent dans le droit chemin : les tests sont son filet.
- La colonne « contexte » grossit à chaque tour, et à chaque tour on renvoie tout.

> Le code généré est une **proposition plausible**, pas une vérité.

La vue d'ensemble de l'agent de code (context window, tools, permissions) ouvre le module 2 : voir le handout « Cadrer avant de coder ».

À retenir

1. Le modèle **prédit**, le harness **agit**.
2. Ce qui sort de la boucle est **plausible** ; c'est le contexte (et les tests) qui le rendent juste.
3. Le modèle relit toute la session à chaque appel : ce qui y traîne compte.
4. Un agent, c'est un modèle qui choisit lui-même ses tools, en boucle, jusqu'à un objectif.

Mots-clés : LLM, token, context window, harness, tools, agent, boucle agentique, hallucination, permissions — voir le glossaire.

FICHE MODULE 2

Cadrer avant de coder

L'agent ne sait que ce qu'on lui donne : cette fiche reprend le module 2 dans l'ordre des slides, et dit quoi lui donner avant qu'il touche au code.

La context window d'un agent de code

Module 2 · slide « La context window d'un agent de code »

C'est **tout** ce que le modèle voit quand il répond :

Bloc	Ce qu'il contient	Chargé
System prompt	les consignes du harness : rôle, règles de sécurité, format des réponses	à chaque appel
Tool definitions	la description de chaque tool exposé (lire, écrire, lancer une commande, MCP)	à chaque appel, avant même le premier message
Instructions projet	le fichier d'instructions du dépôt et les skills chargées : conventions, commandes, pièges	à chaque session (les skills, à la demande)
Conversation	vos messages, les réponses, les corrections et les décisions	renvoyée à chaque appel
Tool calls + résultats	fichiers lus, sorties de commandes, logs d'erreur, résultats de tests	le bloc qui grossit le plus vite
Espace libre	la place qui reste avant la compaction	quand elle rétrécit, la qualité baisse et le coût monte

> Chaque tool exposé, chaque fichier lu, chaque sortie de commande **consomme du budget**.

Une demande ciblée donne exactement ce dont l'agent a besoin, ni plus ni moins. À éviter : « Ajoute la pagination à l'API » en collant tout le dépôt. À la place : l'objectif, les fichiers clés, l'invariant et la commande de vérification.

Plus n'est pas mieux

Module 2 · slide « Plus n'est pas mieux »

Quatre états de la context window :

Situation	État	Pourquoi
« Explique-moi cette regex », dans un chat neuf	**simple**	attention concentrée
L'agent a lu 14 fichiers et lancé 3 fois les tests	**chargée**	les tools dominent la demande

Situation	État	Pourquoi
Tour 40, objectif changé deux fois, log de 3 000 lignes	**polluée**	crédible mais faux : c'est le **context rot**
Session neuve, demande cadrée, plan validé	**préparée**	moins d'exploration à l'aveugle

- **Lost in the middle** : début et fin sont mieux lus que le milieu. L'essentiel en tête (objectif, invariants, instructions projet).
- **Compaction** : quand l'espace libre se réduit, le harness résume la session, et on ne choisit pas ce qui est gardé.
- **Session polluée** : corriger par-dessus aggrave. On extrait ce qui a de la valeur (une note de reprise), on ouvre une session neuve, on recharge la demande initiale et la note.
- **Ouvrir la context window** : regarder l'occupation du contexte et les tool calls ; demander « Quels fichiers as-tu lus ? Sur quelle hypothèse pars-tu ? » ; élaguer ce qui ne sert plus.

> La bonne question : **tout ce qui reste ici aide-t-il encore la tâche du jour ?** Sinon, une **session neuve**.

Vue d'ensemble : l'agent de code

Module 2 · slide « Vue d'ensemble : l'agent de code »

Le trajet d'une demande, du haut vers le bas :

1. **Toi** : tu demandes, tu valides aux points clés, tu relis le diff.
2. **Le harness** assemble la context window (les blocs ci-dessus) et l'envoie au LLM à chaque appel.
3. **Le LLM** prédit la suite : soit il répond, soit il écrit une **demande d'action** (« lis bookings.controller.ts », « lance npm test »).
4. **Le harness** vérifie les **permissions**, exécute le tool et remet la sortie dans la context window.
5. La boucle tourne jusqu'à l'objectif.

Tools, exécutés par le harness	Pour…
Fichiers	lire la code base, morceau par morceau
Terminal	tests, git
Serveurs MCP	tickets, base, documentation

> Le LLM est au centre, mais il ne touche à rien : tout ce qu'il « sait » vient de la context window, tout ce qu'il « fait » passe par les tools et les permissions.

Instructions projet (`AGENTS.md`)

Module 2 · slide « Instructions projet (AGENTS.md) »

Un fichier **versionné** à la racine, **chargé à chaque session** : conventions, commandes, pièges. **Court**, avec des règles **vérifiables**.

Vague	Concret
« Formate proprement »	« Indentation 2 espaces »
« Teste tes changements »	« npm test avant chaque commit »
« Range le code »	« Handlers dans src/api/handlers/ »

- **Court** : quelques centaines de lignes au plus ; au-delà, l'adhérence chute. Chaque ligne est relue à chaque appel : elle doit mériter sa place.
- C'est du **contexte**, pas une configuration imposée : l'agent essaie de le suivre. Des règles spécifiques et non contradictoires sont mieux respectées.
- `AGENTS.md` est un format ouvert, lu par la plupart des harness ; certains ont leur propre nom de fichier (voir « Où ça se règle dans mon outil »).

Le générer, puis le relire

1. **L'agent analyse le dépôt et propose** un premier jet.
2. **Relire** : les concepts métier sont-ils bien identifiés ? les invariants captés ? quelque chose d'halluciné ?
3. **Élaguer** ce qui se déduit du code : l'arborescence et la stack, l'agent les retrouve tout seul.
4. **Ajouter les pièges connus** : votre expertise.

> La génération donne un **point de départ**. Votre expertise **comble les trous**, surtout les pièges.

Une demande initiale bien cadrée

Module 2 · slide « Une demande initiale bien cadrée »

- Le **premier message** de la session : il oriente **toute la suite**.
- **Court et relu** : objectif, fichiers, invariants, vérification.
- Les noms sont des **ancres** : un chemin, une commande, un invariant orientent l'attention vers tout un bloc de code.
- Les **invariants** réduisent les sorties plausibles mais mauvaises.

```text
Pagine GET /bookings par curseur.
Pars de bookings.controller.ts et de shared/pagination.
Fais évoluer le contrat OpenAPI, garde le schéma Page.
Fini quand npm test -- bookings passe.
```

Dans un code inconnu, on ne sait pas l'écrire seul : l'agent explore et propose les fichiers, commandes et pièges, le dev corrige. Pour une **tâche longue**, on écrit la demande dans un **fichier**, rechargé à chaque session : fichiers clés, commandes, invariants, pièges connus.

Planifier avant de coder

Module 2 · slide « Planifier avant de coder »

L'erreur n° 1 : se jeter **directement dans la génération**. Le **mode plan** des harness explore et propose, sans modifier de fichier.

Étape	Ce qui se passe	Exemple
1. **Besoin flou**	le ticket tel qu'on le formule en une phrase	« Pagine /bookings. »
2. **Plan relu**	l'agent propose la demande cadrée et les étapes ; vous les corrigez et les validez	4 étapes vérifiables, réponse `{ items, nextCursor }`, contrat OpenAPI mis à jour, schéma Page inchangé
3. **Code**	généré à partir du plan validé, pas du besoin flou	controller + tests, une étape à la fois

- Un bon plan : définir le problème, découper en **étapes vérifiables** (pour chacune, on sait dire « c'est fait » : un test qui passe, un fichier qui existe), identifier les contraintes (contrat d'API, sécurité, performance).
- Corriger un plan coûte quelques secondes ; corriger le code produit à partir d'un mauvais plan coûte une session.

> On demande d'abord un **plan**, on le corrige, *puis* le code : la sortie d'une étape devient le **contexte** de la suivante.

Niveaux d'autonomie

Module 2 · slide « Modes de pilotage : monter l'autonomie »

Tous les harness proposent les **mêmes niveaux d'autonomie**, sous des noms différents : mode plan, demander, accepter les éditions, tout autoriser…

Niveau d'autonomie	L'agent…	Pour…
Observer sans agir	lit, cherche, propose un plan	explorer un code inconnu
Demander à chaque action	attend ton accord à chaque édition	apprendre à lire ce qu'il propose
Agir dans le périmètre	modifie les fichiers, demande pour le reste	dérouler un plan validé
Aller au bout seul	enchaîne sans rien demander	un environnement isolé : conteneur, worktree

- Un agent qu'on interrompt à chaque commande coûte plus cher qu'il ne rapporte. À l'inverse, monter l'autonomie sans borner les droits expose à tout ce qu'il peut toucher.

- Progression saine : approbation systématique au début, puis on lit le diff plutôt que chaque commande, et jamais « aller au bout seul » sans isolation (conteneur, VM).
- Autonomie et périmètre de droits sont **deux réglages distincts**.

> On monte d'un niveau quand on fait confiance au **périmètre**, avec un **filet** : tests, commits fréquents, diff relu.

À retenir

1. **Instructions projet** : chaque session, courtes, vérifiables.
2. **Demande initiale** : la tâche du jour, avec objectif, fichiers, invariants, vérification.
3. **Plan** : des étapes vérifiables, relues avant le code.
4. **Autonomie** : réglée par niveaux, avec un filet.
5. **Plus n'est pas mieux** : une tâche, une session ; ouvrir la context window, élaguer.

Mots-clés : context window, system prompt, tool definitions, instructions projet, demande initiale, invariant, plan d'exécution, autorégression, niveau d'autonomie, permissions, human-in-the-loop, context rot, lost in the middle, compaction — voir le glossaire.

FICHE MODULE 3

Produire et vérifier

Avancer par petits pas sans perdre la maîtrise du code : la boucle de production, la relecture et les tests.

Feature par feature : la boucle

Module 3 · slide « Feature par feature : la boucle »

Préparer	Livrer, puis feature suivante
1. Lire le ticket	4. Implémenter une étape
2. Faire comprendre le code	5. Tester et relire
3. Demande et plan validés	6. Commit : une étape = un commit (vert et relu, sinon retour à 4)
	7. Étape suivante du plan
	8. Mettre à jour la doc projet

- **Une étape = un commit** : chaque commit est un point de retour propre. Si une étape tourne mal, on revient au commit précédent au lieu de corriger par-dessus.
- **Étape 8** : ce qu'on a appris (un piège, une commande) remonte dans les instructions projet ou dans une skill.

Rewind plutôt qu'empiler

Module 3 · slide « Rewind plutôt qu'empiler »

Empiler	Rewind
Prompt flou	Prompt flou
Code faux	Code faux
« Non, pas comme ça… » : on corrige par-dessus	On édite l'entrée : on revient en arrière
Context window plus sale	Nouvelle trajectoire propre

Empiler, c'est garder la réponse fausse **et** la correction dans tous les envois suivants. En pratique : éditer son message et relancer, ou revenir à un point antérieur de la session (certains harness restaurent aussi les fichiers). Sinon : revenir au dernier commit et ouvrir une nouvelle session avec le prompt corrigé.

> Traitez une mauvaise sortie comme un **message du compilateur** : on corrige l'entrée, on relance.

Git, le filet de l'agent

Module 3 · slide « Git, le filet de l'agent »

Réflexe	Pourquoi	Exemple
Une branche par tâche	le travail de l'agent reste à part jusqu'à la revue ; on peut la jeter	`feat/bookings-pagination`
Un commit par étape	vert et relu : chaque commit est un point de retour	étape 2 du plan, tests verts
Revenir, pas réparer	revenir au dernier commit est déterministe ; « défais ce que tu as fait » ajoute des tokens dans une context window déjà mal orientée	`git restore .` ou `git reset --hard` sur la branche de la tâche
Push et merge : un humain	push, force et merge sont irréversibles ou partagés	l'agent ne pousse, ne force et ne merge jamais seul

- Le rewind et git se combinent : une étape ratée, on revient au dernier commit **et** on relance une session propre.
- Le jour où l'on veut deux agents en même temps, chacun a besoin de sa branche **et** de son répertoire de travail : c'est le **git worktree**, vu dans la formation Avancé.

> Le **rewind** défait la conversation, **git** défait le code.

Relire le code généré

Module 3 · slide « Relire le code généré »

Le code de l'agent se relit comme celui d'un **contributeur inconnu**. La grille, à cocher sur chaque diff :

	Axe	La question
☐	**Correction**	Résout-il vraiment le problème du ticket ?
☐	**Cas limites**	Liste vide, `null`, dernière page, grandes valeurs ?
☐	**Sécurité**	Entrées validées, autorisations correctes ?
☐	**Performance**	Requêtes N+1, boucles, fuites mémoire ?
☐	**Lisibilité**	Compréhensible dans 6 mois, sans sur-ingénierie ?
☐	**Dépendances**	Les imports et les API **existent-ils vraiment** ?

Faites aussi relire l'agent par lui-même, de préférence dans une session neuve :

```text
Relis le diff que tu viens de produire :
cas limites, sécurité, perf.
```

> Le vrai danger : l'**incompétence cachée**, du code qui a l'air bon. L'agent peut se relire, mais c'est **vous** qui décidez.

Deux natures de tests, deux publics

Module 3 · slide « Deux natures de tests, deux publics »

	Tests de construction	Tests fonctionnels
À qui ils servent	à **l'agent**, pour se contrôler pendant qu'il code	à **l'équipe**, pour savoir ce que fait le produit
Ce qu'ils testent	l'implémentation, unité par unité	le **comportement observable** de l'API
Durée de vie	jetables	durables : ils *sont* la spec exécutable

- Un test fonctionnel est un contrat : « GET /bookings renvoie un nextCursor tant qu'il reste des résultats ». C'est celui-là qu'on relit ligne à ligne.
- **Faux succès** : un agent peut affaiblir ou supprimer un test pour le faire passer. On relit les tests comme le code.
- Le coût de production d'un test a chuté, le coût de maintenance n'a pas bougé.

> **En fin de feature**, là où les deux se recouvrent, **le test fonctionnel gagne** : on supprime le doublon.

Brownfield : comprendre et figer avant de modifier

Module 3 · slide « Brownfield : comprendre et figer avant de modifier »

Pas tout le code d'un coup : 500 000 lignes ≈ des millions de tokens, ça ne rentre pas. L'agent **navigue** (recherche, lecture ciblée) ; on attaque **par morceaux** : un module, un endpoint.

Faire comprendre : demander à l'agent d'expliquer le flux de GET /bookings, de la route à la base, avant de le modifier. Si l'explication est fausse, le code le serait aussi.

Figer le comportement actuel avec des tests de caractérisation :

```text
Avant de toucher à BookingsController,
écris des tests qui capturent le
comportement ACTUEL de GET /bookings,
même s'il est faux.
```

Ces tests détectent le **changement**, pas la correction : forme de la réponse, tri, filtres. Si la pagination casse le tri, un test le dit. Un test de caractérisation qui fige un bug est signalé comme tel.

> **Faire comprendre avant de faire modifier** : une explication relue vaut mieux qu'un diff incompris.

Revue IA : donner le contexte du diff

Module 3 · slide « Revue IA : donner le contexte du diff »

Donner à l'agent relecteur	Pourquoi
Le diff complet, avec du contexte autour	voir la fonction entière, pas trois lignes
Le ticket, la demande initiale, le plan	juger le code sur l'intention
Les conventions (instructions projet)	relever les écarts propres à l'équipe
La grille « Relire le code généré »	chercher les défauts, pas complimenter

- Plus de lignes de contexte autour du changement réduit les faux positifs : par exemple `git diff -U10` au lieu des 3 lignes par défaut.
- Une **session neuve** fait un meilleur relecteur que celle qui a écrit le code : elle n'a pas son historique de décisions.

> La revue IA **aide** l'humain, elle ne **décide** pas : c'est un humain qui valide avant le merge.

À retenir

1. Petits pas, **un commit par étape**.
2. **Rewind** plutôt qu'empiler ; **git** pour revenir en arrière dans le code.
3. Des **tests** comme filet et comme spec, relus comme le code.
4. Une **relecture humaine** avant le merge.

Mots-clés : rewind, branche, commit, autorégression, brownfield / greenfield, tests de caractérisation, revue IA, hallucination, human-in-the-loop — voir le glossaire.

FICHE MODULE 4

Étendre l'agent

Brancher l'agent sur le reste du SI, et capitaliser ce qui marche, sans alourdir chaque appel.

Apporter le contexte de l'entreprise

Module 4 · slide « Apporter le contexte de l'entreprise »

Deux façons de donner à l'agent ce que **le dépôt ne contient pas** :

	RAG : vos documents	Tools / MCP : vos applications
Apporte	de la connaissance : ADR, docs d'architecture, wiki, conventions d'API	des données vivantes et la capacité d'agir : tickets, base, CI, navigateur
Exemple	« Quelle est notre convention de pagination ? »	« Lis le ticket BOOK-142, puis lance les tests e2e dans le navigateur. »
Vigilance	des documents à jour, sinon des réponses périmées… mais sourcées	des droits limités : l'agent agit avec vos accès

- **RAG** : les documents sont indexés à l'avance ; un tool trouve les passages pertinents ; seuls ces passages entrent dans la context window ; le modèle s'appuie dessus et cite ses sources. Pour le code, la recherche dans le dépôt joue déjà ce rôle ; le RAG sert pour ce qui n'est pas dans le dépôt.
- **MCP** (Model Context Protocol) : la prise standard des tools, l'« USB-C » des agents. Un serveur écrit une fois marche dans tous les harness compatibles, en local ou à distance. MCP n'apporte aucune « intelligence » : c'est de la plomberie standardisée.
- Le modèle choisit un tool sur sa **description** : nom, description et paramètres de chaque tool sont envoyés à chaque appel. Une mauvaise description = un tool mal utilisé ou ignoré.

> Dans les deux cas, l'agent ne **sait** rien de plus : on lui **donne** le bon contexte, au bon moment.

MCP : ce que vous exposez en branchant un serveur

Module 4 · slide « MCP : ce que vous exposez en branchant un serveur »

Brancher un serveur, c'est laisser le modèle **décider seul** quelles données partent chez un tiers. Trois questions avant chaque branchement :

Question	Ce qu'on vérifie
Qui l'exécute ? Local ou distant	Le local reste sur la machine. Le distant reçoit ce que le modèle lui envoie.
Qui l'authentifie ? Quel jeton, quels droits	Lecture, écriture, suppression : jamais plus large que la tâche.
Qui le maintient ? Officiel ou fork	Un serveur non maintenu est un risque ouvert.

Et ce qu'il coûte : chaque serveur ajoute ses tool definitions à **chaque** appel, avant même le premier message. N'activez que ceux que vous utilisez **vraiment** pour ce projet. Une commande en ligne déjà installée, documentée dans une skill, coûte souvent moins de contexte qu'un serveur MCP complet.

Une bonne consigne devient une skill

Module 4 · slide « Une bonne consigne devient une skill »

Un **savoir-faire emballé** :

- un **dossier** : un `SKILL.md`, plus des scripts, références et gabarits ;
- chargée **à la demande**, quand la tâche correspond : la **progressive disclosure** (l'agent ne voit d'abord que le nom et la description) ;
- la **description** dit quoi et quand : c'est elle qui déclenche ;
- testée en **session vierge** : une autre session la suit sans coaching.

Exemple, la skill « tests-api » :

```text
tests-api/SKILL.md

name: tests-api
description: Écrire ou compléter les tests
  d'un endpoint. À utiliser dès qu'on
  ajoute ou modifie une route.

- Un test par cas limite : liste vide,
  dernière page, curseur invalide.
- Données via tests/factories.
- Base de test réelle, pas de mock.
```

- On n'y met que ce que l'agent **ne peut pas déduire** ; quelques centaines de lignes au plus ; pas de chemins propres à un OS, de versions figées ni d'informations périmées.
- Un **script** embarqué donne le même résultat à chaque fois, là où la même logique en prose est réinterprétée ; et il n'entre pas dans la context window tant que l'agent ne l'ouvre pas.
- Si la skill ne se déclenche pas en session vierge, c'est presque toujours la **description** qu'il faut réécrire.

> Une skill ne pèse dans la context window **que quand elle sert**.

Où ranger le contexte durable ?

Module 4 · slide « Quiz : où ranger le contexte durable ? »

La règle qui tranche : **à quelle fréquence** le contenu doit-il être chargé ?

Où	Chargé	Exemple
Instructions projet	à chaque session	« Les tests se lancent avec npm test, jamais sur la base de prod » ; « TypeScript strict, handlers dans src/api/handlers »

Où	Chargé	Exemple
Demande initiale	pour une tâche	« Pour ce ticket : garder le schéma Page de /rooms »
Skill	à la demande, quand la tâche correspond	« Comment écrire un test d'API selon nos conventions » ; « La procédure de migration de schéma, avec son script »
Note de reprise	pour reprendre une session interrompue	« Étapes 1 à 3 faites, l'étape 4 bloque sur le curseur »

- Piège : « jamais sur la base de prod » pourrait sembler une skill ; elle doit valoir à chaque session, donc instructions projet.
- Tous ces fichiers sont **versionnés et relus comme du code** : ils entrent dans la context window de toute l'équipe.

> Plus c'est **chargé souvent**, plus c'est **court** : à chaque session, pour une tâche, à la demande.

À retenir

1. Tools et MCP branchent l'agent sur le SI, mais chaque branchement **expose** quelque chose et **pèse** dans chaque appel.
2. Une skill est un savoir-faire **chargé à la demande** ; sa description la déclenche.
3. Une consigne que tu répètes **trois fois** mérite une skill, testée en session vierge.

Mots-clés : RAG, grounding, tools, MCP, tool definitions, skill, progressive disclosure, instructions projet, demande initiale — voir le glossaire.

FICHE MODULE 5

Garder la main

Ce qui peut mal tourner avec un agent de code, dans l'ordre des slides du module 5 : des coûts qui s'emballent, des données qui fuient. La context window polluée (« Plus n'est pas mieux ») est au module 2.

Coûts : le token se paie

Module 5 · slide « Coûts : le token se paie »

À chaque tour, le harness renvoie tout le contexte : dans la trace du module 1, 7 tours de 12 k à 21 k tokens, soit **118 k tokens d'entrée cumulés** pour une tâche simple (chiffres illustratifs). Le **cache** du fournisseur sert le début inchangé du contexte à moindre coût ; changer de modèle ou de MCP en cours de route l'invalide.

5 réflexes

1. **1 tâche = 1 session** : moins d'historique renvoyé.
2. **Être précis** : une demande cadrée, moins d'exploration.
3. **Couper les MCP inutiles** : chacun pèse dans le contexte.
4. **Garder le même modèle** : sinon le **cache** est perdu.
5. **Demander du concis** : moins de tokens en sortie.

> Le token se paie, en **entrée** comme en **sortie** : le préfixe déjà vu est **renvoyé à chaque tour**.

5 risques, 5 garde-fous

Module 5 · slide « 5 risques, 5 garde-fous »

Risque	Ce qui arrive	Exemple réel	Garde-fou
Hallucination	une API ou un paquet inventés, mais plausibles	un paquet inventé par l'IA, publié ensuite par un tiers	tests, typage, relecture
Prompt injection	un contenu piégé détourne l'agent	une issue publique qui fait fuiter un dépôt privé	accès de l'agent limités
Fuite de données	code, secrets ou données sortent de l'entreprise	du code propriétaire collé dans un chatbot	outils validés, pas de secret
Action destructrice	l'agent supprime ou écrase ce qu'il ne fallait pas	une base de production effacée par un agent	sandbox, humain qui valide
Coûts	une consommation qui s'emballe	des clés volées, des factures à 5 chiffres	plafonds, suivi

- **Prompt injection** : pour le modèle, tout est du texte, donnée et consigne se confondent. L'instruction peut être cachée dans un README, une issue, une page web, une réponse MCP. Aucun garde-fou ne bloque 100 % des injections : l'architecture (permissions, isolation) borne le risque.

- **Faux succès** : « tous les tests passent »… après avoir supprimé le test. On relit le diff et les tests.
- **Secrets** : jamais de `.env`, de jeton ou de mot de passe dans le contexte. Ce qui entre dans la context window part chez le fournisseur du modèle.
- **Dépendances** : vérifier qu'un paquet existe **et** qu'il est le bon.

Le trio létal

Module 5 · slide « 5 risques, 5 garde-fous » (encadré)

Données privées + **contenu non fiable** + **sortie vers l'extérieur** = exfiltration possible. L'issue piégée coche les trois cases : un agent qui lit les issues publiques, accède au dépôt privé et peut commenter.

> **Casser au moins une branche** : pas d'accès au privé, ou pas de droit d'écrire vers l'extérieur.

Une consigne « ne lance jamais de commande destructrice » dans les instructions projet n'est qu'une suggestion : la règle vit dans le **harness** (permissions, isolation), pas dans le prompt.

2 règles d'or

Module 5 · slide « 2 règles d'or »

Règle	Ce qu'elle couvre	Le test
1. Code et données sensibles : outils validés	Rien de confidentiel hors des harness et modèles validés par l'entreprise : code client, données de prod, secrets (`.env`, jetons).	L'enverriez-vous par email à un inconnu ?
2. Un humain valide avant le merge	Obligatoire pour ce qui est irréversible : merge, déploiement, migration, push forcé. Le code généré est le vôtre.	Qui signe si c'est faux ?

- **Shadow AI** : un abonnement personnel n'a pas les garanties d'un compte entreprise (rétention, entraînement sur les données, localisation).
- En mission chez un client, ce sont **les règles du client** qui s'appliquent : vérifier avant la première session.
- La mention « co-écrit avec l'IA » dans un commit ne transfère aucune responsabilité.

> Ces deux règles couvrent l'essentiel des risques : la **fuite** d'un côté, l'**erreur** de l'autre. C'est le **human-in-the-loop**.

À retenir

1. **Une tâche, une session.** Jamais de secret dans le contexte.
2. Trio létal : **casser une branche**.
3. Le token se paie ; les règles de l'entreprise s'appliquent.

Mots-clés : cache, plafond de dépense, prompt injection, fuite de données, trio létal, shadow AI, human-in-the-loop — voir le glossaire.

Et après ?

Clôture · slide « Et après ? »

Niveau d'autonomie	Ce qu'il demande	Où
Observer sans agir	une demande cadrée, un plan relu	Foundation
Demander à chaque action	savoir lire ce que l'agent propose	Foundation
Agir dans le périmètre	tests, un commit par étape, diff relu	Foundation
Aller au bout seul	un harness configuré, un environnement isolé, un juge	formation Avancé

La formation **Développeur IA — Avancé** reprend au cran 4 : un dev, plusieurs agents, un système. Économie de l'agent (prompt caching), harness configuré (hooks, sandbox), ingénierie du contexte, spec-driven, patterns agentiques (Plan-and-Execute, GOAP, agent as a judge, best-of-N), et plusieurs agents en parallèle sur des **worktrees**, pilotés depuis un **ADE**. Conseil : pratiquer d'abord les réflexes de la Foundation sur un vrai ticket.

SUPPORT D'EXERCICE

Les exercices

Les trois exercices des slides : optionnels, indépendants et courts, à faire avec ton propre harness sur le kit d'exercices.

Le kit se télécharge depuis la page de la formation (« Kit de code (.zip) »). Chaque exercice part du kit tel quel : tu peux en faire un, deux, trois… ou aucun, la formation se suit sans. Les consignes sont écrites en **intention** : chacun les réalise avec le mécanisme de son harness (voir « Où ça se règle dans mon outil »).

Exercice 1 — Du ticket au plan (15 min, optionnel)

Module 2 · slide « Exercice 1 — Du ticket au plan »

Concept : cadrer avant de coder.

Demande à ton agent…

1. **sans rien modifier**, de proposer la **demande initiale** et un **plan** pour `TICKET.md` ;
2. … puis **corrige-les**.

```text
Lis TICKET.md. Sans rien modifier, propose la demande initiale
(fichiers clés, commandes, invariants, pièges) et un plan
en étapes vérifiables.
```

Réussi quand…

- **aucun fichier** n'a été modifié ;
- au moins **une correction** vient de toi.

Exercice 2 — Petits pas, tests, revue (20 min, optionnel)

Module 3 · slide « Exercice 2 — Petits pas, tests, revue »

Concept : produire et vérifier.

Demande à ton agent…

1. d'implémenter **une étape** du plan, **avec ses tests** ;
2. … puis **relis le diff** avec la grille « Relire le code généré », et fais-le relire par l'agent.

Pas de plan ? Pars de `docs/plan-de-secours.md`.

```text
Fais uniquement l'étape 1 du plan, avec ses tests.
```

Réussi quand…

- les **tests passent** ;
- tu as repéré **un défaut** de l'agent.

Exercice 3 — Ma première skill (10 min, optionnel)

Module 4 · slide « Exercice 3 — Ma première skill »

Concept : capitaliser.

Demande à ton agent…

1. d'écrire en `SKILL.md` une consigne que tu répètes ;
2. … puis teste-la en **session vierge**.

Pas d'idée ? « Nos tests utilisent `tests/factories`, sans mock. » Modèle : la skill « tests-api » du handout « Étendre l'agent ».

```text
Écris en SKILL.md cette consigne : [la consigne].
Nom, description (quand l'utiliser), consignes.
```

Réussi quand…

- la skill **se déclenche** en session vierge.

DÉMOS À REJOUER

Les démos

Les démos jouées par le formateur pendant la formation, pour les rejouer seul, que tu aies été présent ou non. Les deux premières demandent un assistant de chat, la troisième un agent de code et le kit d'exercices (installation dans son `README.md`).

Le texte découpé en tokens

Module 1 · slide « Le texte découpé en tokens » · 1 min, optionnelle

Ce que ça montre : le modèle ne lit pas des mots mais des **tokens**. Les mots courants tiennent en un token ; les mots rares et les identifiants sont découpés ; le français consomme plus que l'anglais.

À rejouer

1. Ouvre un visualiseur de tokens en ligne (par exemple le *tokenizer* d'OpenAI).
2. Tape un nom de classe ou de table de ton projet, et regarde en combien de morceaux il est découpé.
3. Tape la même demande en français puis en anglais, puis une ligne de code, et compare les nombres de tokens.

Ce qu'on observe : les identifiants et le code sont découpés finement, et l'indentation compte aussi. Un fichier de 2 000 lignes lu par l'agent, ce sont des dizaines de milliers de tokens dans la context window : on facture et on limite en tokens, pas en lignes.

La souris mange…

Module 1 · démo « La souris mange… » · 3 min

Ce que ça montre : le modèle choisit la suite **la plus probable**, pas la plus vraie. Relancer change la forme, rarement l'idée ; un seul mot ajouté déplace toute la suite.

À rejouer

1. Avant de lancer, devine toi-même la réponse.
2. Dans une conversation vide, colle :

```text
La souris mange …
Complète la phrase en une ligne.
```

3. Relance 2 ou 3 fois (regenerate) : note ce qui change et ce qui reste.
4. Ajoute **un seul mot** et relance : « La souris **d'ordinateur** mange… », puis « La souris **de laboratoire** mange… ».
5. Compare les réponses côte à côte.

Ce qu'on observe : la première réponse est celle que presque tout le monde devine, parce que c'est la suite la plus fréquente dans les textes, pas parce qu'elle est vraie (une souris mange surtout des graines). Les relances changent la formulation, pas l'idée. Le mot ajouté change toute la réponse. Même chose pour le code : après `res.status(`, le modèle écrit le code de retour le plus fréquent, pas celui de ton contrat d'API.

Pour aller plus loin (3 min) : rendre la variance visible.

1. Demande « Écris une phrase sur une souris. » et relance 3 à 5 fois.
2. Change très peu de mots : ajoute « concise », puis « minimal ».
3. Ajoute un intrus : « Victor Hugo », « Fortnite », un bout de C++.

Le même prompt donne plusieurs trajectoires, et l'intrus contamine la sortie. Dans un dépôt, un nom de fichier, un commentaire obsolète ou une vieille stack trace orientent le code produit de la même façon.

Une trace d'agent, commentée

Module 1 · slide « Une trace d'agent, commentée » · 3 min, optionnelle

Ce que ça montre : un agent découvre le code comme nous (il cherche, il lit), demande l'accord avant d'écrire, et ce sont les tests qui le remettent dans le droit chemin. À chaque tour, tout le contexte est renvoyé.

À rejouer, sur le kit installé (`npm install` , `npm test` vert), avec l'affichage des appels de tools activé dans ton harness :

```text
Ajoute la pagination par curseur sur GET /bookings, avec ses tests.
```

Ce qu'on observe, tour par tour :

- ce que l'agent cherche et lit avant d'écrire la moindre ligne ;
- le moment où le harness demande ton accord pour modifier un fichier ;
- ce qui se passe quand un test échoue ;
- la jauge de contexte, qui grossit à chaque tour.

Compare avec la trace de la slide (7 tours, de 12 k à 21 k tokens) : l'ordre des étapes est souvent proche, les chiffres varient. Pour revenir à l'état initial du kit ensuite, si tu as fait un premier commit : `git restore .` (ou `git stash`).

Les exercices, montrés en démo

Si le temps manque, le formateur montre un exercice en démo au lieu de le faire faire. Le déroulé est celui de l'exercice, dans le handout « Les exercices » :

- Exercice 1 — Du ticket au plan : l'agent en mode plan sur le ticket du kit, et ce que son plan rate ;
- Exercice 2 — Petits pas, tests, revue : une seule étape du plan, ses tests, la relecture du diff, le commit ;
- Exercice 3 — Ma première skill : la skill « tests-api », et la preuve qu'elle se déclenche en session vierge.

FICHE OUTILS

Où ça se règle dans mon outil

La formation parle de concepts ; cette fiche donne, pour quatre harness courants, l'endroit où chaque concept se règle.

> **Vérifié sur la documentation officielle des éditeurs le 21 octobre 2026 ; les outils changent vite.** Les noms de modes, les commandes et les chemins bougent d'une version à l'autre : en cas de doute, la documentation du jour fait foi. « — » signale l'absence d'équivalent documenté.

Les formats ouverts passent d'un outil à l'autre : `AGENTS.md` (instructions projet), `SKILL.md` (skills) et MCP (tools). Une équipe mixte peut versionner un seul jeu de fichiers, chacun garde son harness.

Les fichiers de contexte

Slides « Instructions projet (AGENTS.md) » (module 2), « MCP : ce que vous exposez en branchant un serveur » et « Une bonne consigne devient une skill » (module 4)

Concept	GitHub Copilot	Claude Code	Codex	Cursor
Instructions projet	`AGENTS.md` · `.github/copilot-instructions.md` · `.github/instructions/*.instructions.md` ciblés par glob (`applyTo`)	`CLAUDE.md` (projet : `./CLAUDE.md` ; perso : `~/.claude/CLAUDE.md`) · `AGENTS.md` , lu seulement s'il n'y a pas de `CLAUDE.md` (sinon l'importer : `@AGENTS.md`)	`AGENTS.md` (perso : `~/.codex/AGENTS.md`)	`AGENTS.md` · `.cursor/rules/*.mdc`
Générer un premier jet	`/init` (CLI) : écrit `.github/copilot-instructions.md`	`/init`	`/init` : écrit un `AGENTS.md`	`/create-rule` : écrit une règle dans `.cursor/rules`
Skills (`SKILL.md`)	`.github/skills/<nom>/` (lit aussi `.claude/skills/` et `.agents/skills/`) ; perso : `~/.copilot/skills/`	`.claude/skills/<nom>/` ; perso : `~/.claude/skills/`	`.agents/skills/<nom>/` ; perso : `~/.agents/skills/`	`.cursor/skills/` ou `.agents/skills/` (lit aussi `.claude/skills/`) ; perso : `~/.cursor/skills/`
Déclencher une skill	le modèle, sur la description, ou `/<nom>` dans la demande	le modèle, sur la description, ou `/<nom>`	le modèle, sur la description, ou `$<nom>`	le modèle, sur la description, ou `/<nom>`
Serveurs MCP	CLI : `~/.copilot/mcp-config.json` (ou `/mcp add`) · projet : `.mcp.json` ou `.github/mcp.json` · VS Code : `.vscode/mcp.json` (clé `servers` , pas `mcpServers`)	`.mcp.json` à la racine, versionné (ou `claude mcp add`)	`~/.codex/config.toml` ou `.codex/config.toml` du projet, table `[mcp_servers.<nom>]` (ou `codex mcp add`)	`.cursor/mcp.json` (projet) · `~/.cursor/mcp.json` (perso)

Les niveaux d'autonomie

Slides « Planifier avant de coder » et « Modes de pilotage : monter l'autonomie » (module 2)

Niveau d'autonomie	GitHub Copilot	Claude Code	Codex	Cursor
Observer sans agir (mode plan, lecture seule)	mode *plan* (`/plan` , Maj+Tab)	mode *plan*	mode plan (`/plan`) · `sandbox_mode = "read-only"`	mode *Ask* (lecture seule) ou *Plan* (Maj+Tab)
Demander à chaque action	permissions `default` (CLI) · *Manual permissions* (VS Code, par défaut)	mode *Manual* (valeur `default`)	`trust_level = "untrusted"` sur le projet (`approval_policy = "untrusted"` est retiré)	mode *Allowlist* : tout ce qui n'est pas listé demande une approbation

Niveau d'autonomie	GitHub Copilot	Claude Code	Codex	Cursor
Agir dans le périmètre	approbations mémorisées par emplacement (« ne plus demander dans ce dépôt ») · `--allow-tool`	*accept edits* (`acceptEdits`)	préréglage *Auto* (par défaut) : `approval_policy = "on-request"` + `sandbox_mode = "workspace-write"`	mode *Auto-review* : liste blanche, sandbox, puis classifieur
Aller au bout seul	*Autopilot* · *Allow all* (`/yolo` , `--yolo`) · *assisted* (un LLM juge chaque demande)	*auto mode* (classifieur, mode de départ par défaut depuis la v2.1.283) · *bypass permissions*	*Auto-review* (`approvals_reviewer = "auto_review"`) · `--yolo` (ni sandbox ni approbation)	mode *Run Everything* (`/run-everything` en CLI)
Où se déclarent les droits	`--allow-tool` / `--deny-tool` au lancement · approbations mémorisées dans `~/.copilot/permissions-config.json` · VS Code : réglages `chat.tools.*.autoApprove`	`permissions.allow` / `ask` / `deny` dans `.claude/settings.json` , versionné	`~/.codex/config.toml` : `approval_policy` et `sandbox_mode` , réglés séparément · `/permissions` en session	*Settings > Agents > Approvals & Execution* · `.cursor/permissions.json` (consignes de l'Auto-review) · CLI : `permissions.allow` / `deny` dans `.cursor/cli.json`

- Le mode affiché n'est qu'un **défaut** : une autorisation large en configuration fait que l'agent ne demande plus rien. La configuration a le dernier mot.
- « Aller au bout seul » n'est raisonnable que dans un environnement isolé et jetable (conteneur, VM).

Les actions en session

Slides « Rewind plutôt qu'empiler » (module 3), « Plus n'est pas mieux » et « Coûts : le token se paie » (module 5)

Action	GitHub Copilot	Claude Code	Codex	Cursor
Revenir en arrière (rewind, checkpoints)	VS Code : *Restore Checkpoint* sur une demande · CLI : `/rewind` (ou `/undo`), conversation seule ou conversation + fichiers	`/rewind` ou Échap deux fois : revient à un message, restaure le code, la conversation ou les deux	Échap deux fois : édite un message précédent et repart de là ; fichiers : git	*Restore Checkpoint* dans le chat (fichiers) · CLI : `/rewind`
Voir l'occupation de la context window	`/context` · indicateur (petit cercle) dans VS Code	`/context`	`/status` · « % context left » en pied d'écran	anneau de contexte à côté de la saisie (clic : détail par catégorie)
Voir la dépense	`/usage` (CLI : tokens par modèle, AI Credits)	`/usage` (`/cost` en est l'alias)	`/usage`	tableau de bord *Spending*, hors session
Compacter · repartir à zéro	`/compact` · `/clear` (alias `/new`)	`/compact` · `/clear` (alias `/new`)	`/compact` · `/new` (ou `/clear`)	`/summarize` · nouveau chat (CLI : `/clear`)

> Partout, le filet universel reste le même : **revenir au dernier commit** et ouvrir une **session neuve** avec la demande corrigée.

Sources

Documentation officielle des éditeurs, consultée le 21 octobre 2026.

- GitHub Copilot
  - Instructions projet : https://docs.github.com/en/copilot/how-tos/configure-custom-instructions/add-repository-instructions
  - Skills : https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/add-skills
  - CLI (commandes, modes, MCP) : https://docs.github.com/en/copilot/reference/copilot-cli-reference/cli-command-reference
  - Permissions CLI : https://docs.github.com/en/copilot/how-tos/copilot-cli/use-copilot-cli/allowing-tools
  - Context window : https://docs.github.com/en/copilot/concepts/prompting-and-context/context-management
  - VS Code, approbations : https://code.visualstudio.com/docs/agents/run/approvals
  - VS Code, checkpoints : https://code.visualstudio.com/docs/agents/run/review-code-edits
  - VS Code, MCP : https://code.visualstudio.com/docs/agent-customization/mcp-servers

- Claude Code
  - Instructions projet (`CLAUDE.md` , `AGENTS.md`) : https://code.claude.com/docs/en/memory
  - Skills : https://code.claude.com/docs/en/skills
  - Modes de permission : https://code.claude.com/docs/en/permission-modes
  - Permissions : https://code.claude.com/docs/en/permissions
  - MCP : https://code.claude.com/docs/en/mcp
  - Checkpoints : https://code.claude.com/docs/en/checkpointing
  - Commandes : https://code.claude.com/docs/en/commands
- Codex (les liens developers.openai.com/codex redirigent vers learn.chatgpt.com)
  - `AGENTS.md` : https://developers.openai.com/codex/guides/agents-md
  - Skills : https://developers.openai.com/codex/skills
  - Approbations et sandbox : https://developers.openai.com/codex/agent-approvals-security
  - MCP : https://developers.openai.com/codex/mcp
  - Commandes : https://developers.openai.com/codex/cli/slash-commands
- Cursor
  - Règles et `AGENTS.md` : https://cursor.com/docs/rules
  - Skills : https://cursor.com/docs/skills
  - MCP : https://cursor.com/docs/mcp
  - Modes d'exécution : https://cursor.com/docs/agent/security/run-modes
  - Mode Ask : https://cursor.com/help/ai-features/ask-mode
  - Mode Plan : https://cursor.com/docs/agent/plan-mode
  - Checkpoints : https://cursor.com/docs/agent/overview
  - Context window : https://cursor.com/docs/agent/prompting
  - CLI (commandes, permissions) : https://cursor.com/docs/cli/reference/slash-commands
  - Dépense : https://cursor.com/help/models-and-usage/usage-limits

GLOSSAIRE

Glossaire des mots-clés IA

Une phrase par terme : les mêmes mots, avec le même sens, que dans la formation business, puis les termes propres aux développeurs.

Les termes techniques anglais sont gardés (harness, tools, context window, skill) : ce sont ceux que vous entendrez dans les outils et les réunions.

Tronc commun : les bases

Ouverture et module 1

Terme	En une phrase
LLM (Large Language Model)	Modèle entraîné sur d'énormes volumes de texte, qui génère une réponse en prédisant le mot suivant.
Token	Morceau de mot traité par le modèle ; c'est aussi l'unité de facturation (≈ ¾ de mot en anglais).
Hallucination	Réponse fausse mais présentée avec assurance et vraisemblance.
Knowledge cutoff (date de coupure)	Date à laquelle s'arrêtent les données d'entraînement du modèle ; il ignore tout ce qui s'est passé après, sauf s'il fait une recherche web.
Assistant	Le produit qu'on utilise au quotidien (ChatGPT, Copilot, Claude.ai, Gemini) : une interface posée sur un harness, autour d'un modèle.
Harness	Tout le logiciel autour du modèle : consignes, mémoire, tools, enchaînement des étapes. Deux assistants sur le même modèle diffèrent par leur harness.
Tools	Capacités que le modèle peut appeler pendant sa réponse : recherche web, calcul, lecture de fichiers, Jira, Figma… (*tool calling*).
MCP (Model Context Protocol)	Standard pour brancher de nouveaux tools sur un assistant : applications et données de l'entreprise.
Agent	Modèle qui enchaîne lui-même des actions avec des tools, en boucle, jusqu'à atteindre un objectif : c'est la **boucle agentique** (modèle + tools + boucle + objectif, que le harness fait tourner). Les éditeurs utilisent aussi « agent » pour de simples assistants personnalisés.
Boucle agentique (agent loop)	Le cycle demande → le modèle choisit un tool → le harness l'exécute → le résultat entre dans le contexte → on recommence si besoin.

Tronc commun : demander et capitaliser

Ouverture, modules 2 à 4

Terme	En une phrase
Prompt	La demande envoyée au modèle, consignes comprises.
Rewind	Revenir corriger la demande de départ plutôt qu'empiler des corrections dans la conversation.
Instructions personnalisées / assistant personnalisé	Consignes enregistrées une fois et réappliquées : toujours (instructions personnalisées) ou dans un espace dédié (GPT, Projet Claude, agent Copilot).
Skill	Savoir-faire emballé (consignes, modèles, exemples, parfois des scripts) que l'assistant charge **à la demande**, quand la tâche correspond. Plus léger que des instructions permanentes, partageable dans une équipe. Claude parle d'« Agent Skills » ; d'autres assistants ont des équivalents.
Vibe coding	Produire vite un prototype en langage naturel, sans regarder le code ; parfait pour explorer, insuffisant pour livrer.
AI-driven development	Développement logiciel où l'IA écrit beaucoup de code, sous la direction et le contrôle d'ingénieurs.

Tronc commun : le contexte

Modules 1, 4 et 5

Terme	En une phrase
Context window	Tout ce que le modèle « voit » au moment de répondre : consignes, historique, pièces jointes, résultats des tools.
Session	Une conversation ; elle accumule du contexte, utile ou parasite.
Mémoire	Informations que l'assistant réinjecte automatiquement d'une session à l'autre (profil, préférences) ; souvent désactivable.
Grounding	Ancrer la réponse dans des sources vérifiables plutôt que dans la « mémoire » du modèle.
RAG (Retrieval-Augmented Generation)	Technique de grounding : on cherche les bons extraits de documents internes et on les ajoute au contexte avant de répondre.

Tronc commun : risques et coûts

Module 5

Terme	En une phrase
Human-in-the-loop	Un humain qualifié valide la sortie de l'IA *avant* qu'elle soit utilisée.

Terme	En une phrase
Prompt injection	Instructions malveillantes cachées dans un contenu (email, document, page web) qui détournent un assistant.
Fuite de données	Données confidentielles envoyées à un assistant non maîtrisé (version grand public).
Cache	Mise en mémoire du début du contexte pour ne pas le repayer plein tarif à chaque message (il est relu à prix réduit) ; changer de modèle ou de tools en cours de route le perd.
Plafond de dépense (spend cap)	Limite de budget configurée pour éviter les factures surprises.

Termes propres aux développeurs : la machine

Modules 1 et 2

Terme	En une phrase
Agent de code	Agent qui lit, modifie et exécute du code dans un dépôt (Copilot, Claude Code, Codex, Cursor…).
Non-déterminisme	La même demande peut donner des sorties différentes : une réponse est un tirage, pas un calcul.
Autorégression	Le modèle génère token par token, chaque token produit devenant l'entrée du suivant : ce qu'il a écrit oriente ce qu'il écrira.
System prompt	Consignes que le harness place en tête de chaque appel, avant les instructions projet et la conversation.
Tool definitions	Nom, description et paramètres de chaque tool, envoyés au modèle à chaque appel : ils pèsent dans la context window et se lisent comme un prompt.
Permissions	Les actions autorisées à l'agent (lire, écrire, lancer une commande, accéder au réseau).

Termes propres aux développeurs : cadrer, produire, capitaliser

Modules 2 à 4

Terme	En une phrase
Instructions projet (`AGENTS.md`)	Fichier versionné dans le dépôt, chargé à chaque session : conventions, commandes, pièges. C'est l'équivalent dev des instructions personnalisées.

Terme	En une phrase
Demande initiale	Premier message de la session, cadré pour la tâche du jour : objectif, fichiers clés, commandes, invariants, pièges. Pour une tâche longue, on l'écrit dans un fichier rechargé à chaque session.
Invariant	Règle qui doit rester vraie quoi que fasse l'agent (un contrat d'API, une règle métier, un test qui doit passer).
Plan d'exécution	Liste d'étapes vérifiables que l'agent propose avant de coder, et que le dev corrige avant de le laisser faire.
Niveau d'autonomie	Ce que l'agent peut faire sans demander : proposer, modifier après accord, ou agir seul.
Revue IA	Relecture d'un diff par un agent ; elle aide l'humain, elle ne décide pas du merge.
Brownfield / greenfield	Code existant à faire évoluer / projet partant de zéro.
Progressive disclosure	L'agent ne voit d'abord que le nom et la description d'une skill, et ne charge son contenu que quand il en a besoin.

Termes propres aux développeurs : garder la main

Module 5

Terme	En une phrase
Context rot (en business : dérive de contexte)	Dégradation des réponses quand la context window se remplit d'ajouts, de corrections et d'informations périmées ou contradictoires : la session reste crédible mais s'éloigne du besoin initial.
Compaction	Résumé automatique ou demandé de la session pour libérer de la context window.
Lost in the middle	Le modèle exploite moins bien ce qui se trouve au milieu d'un long contexte.
Trio létal	Données privées + contenu non fiable + capacité d'envoyer vers l'extérieur : la combinaison qui rend une fuite possible.
Shadow AI	Usage d'outils d'IA non validés par l'entreprise.